

Spring



Controller - Service - Repository - Database

HTTP

Spring Boot

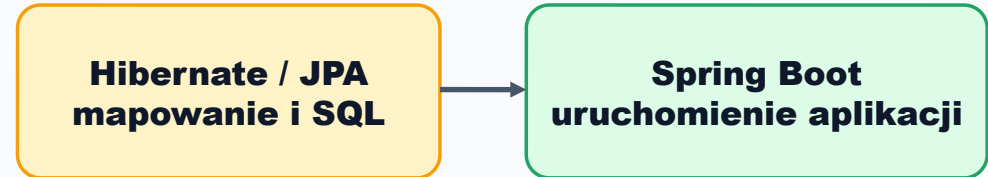
JPA

DB

<https://github.com/umcsuser/demo>

SPRING

Po Hibernate dołożymy warstwę aplikacji webowej



jak udostępnić logikę przez HTTP?

HTTP (Hypertext Transfer Protocol) to protokół komunikacyjny, który umożliwia przeglądarce i serwerowi wymianę danych w internecie.

REST API (Representational State Transfer) to zbiór reguł architektonicznych służących do komunikacji między systemami komputerowymi przez internet, najczęściej z wykorzystaniem protokołu HTTP. Pozwala aplikacjom (klientom) na pobieranie, tworzenie, aktualizowanie i usuwanie danych (zasobów) z serwera, używając standardowych metod, takich jak GET, POST, PUT, DELETE.



Krótką historia Springa

Spring to framework a obecnie cały ekosystem umożliwiający tworzenie aplikacji webowych z wykorzystaniem języka Java.

Jest alternatywą dla JEE (Java Enterprise Edition)

Pomysł na Springa powstał podczas pisania książki Expert One-on-One J2EE Design and Development napisanej przez Roda Johnsona w 2002 roku.

Głównym celem Springa jest umożliwienie tworzenia złożonych systemów w łatwy sposób, z pominięciem modeli programowania wymuszających historyczne rozwiązania.



Kod aplikacji ma skupiać się na logice biznesowej, a nie na ręcznym składaniu infrastruktury.

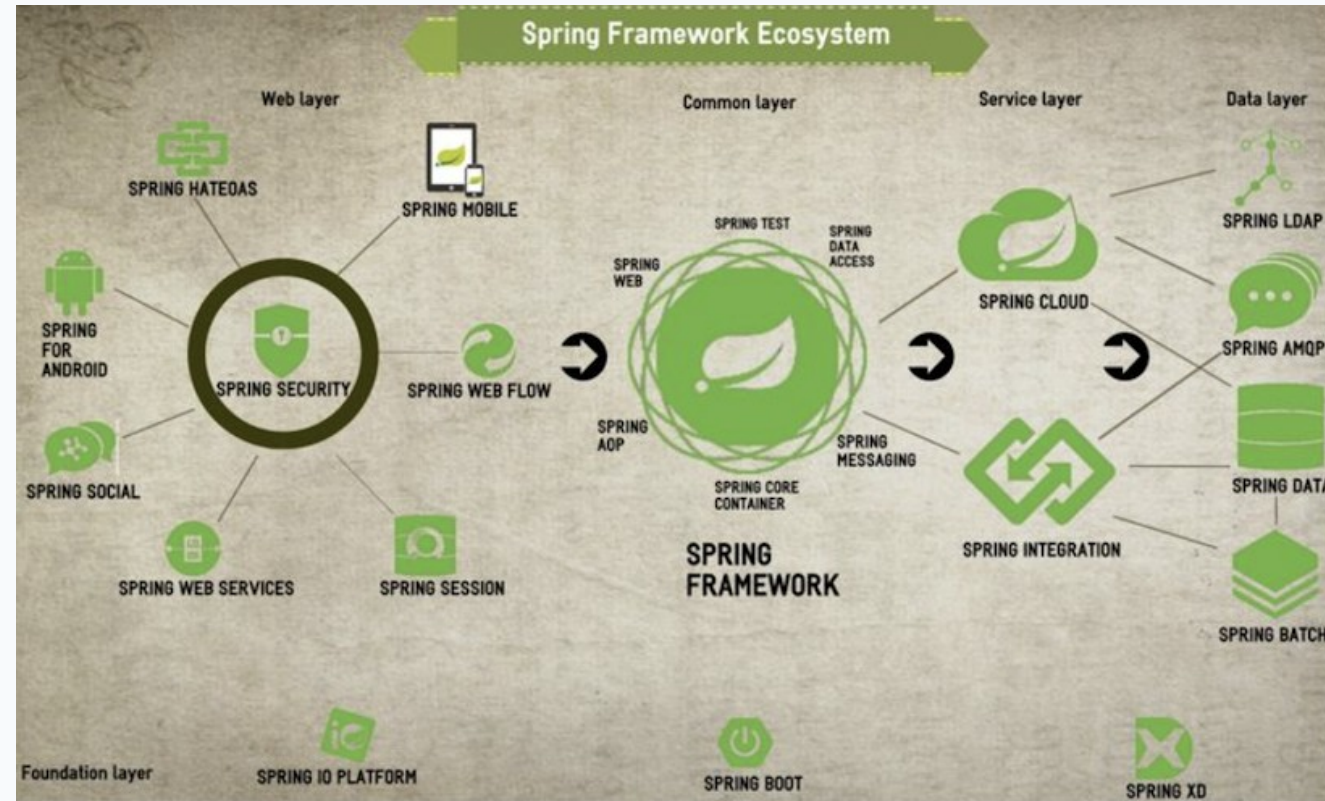
Spring jako ekosystem

To zestaw projektów wybieranych według potrzeb

„Spring” to rodzina projektów. Wybieramy tylko te elementy, które są potrzebne w danej aplikacji.

W przykładzie z zajęć

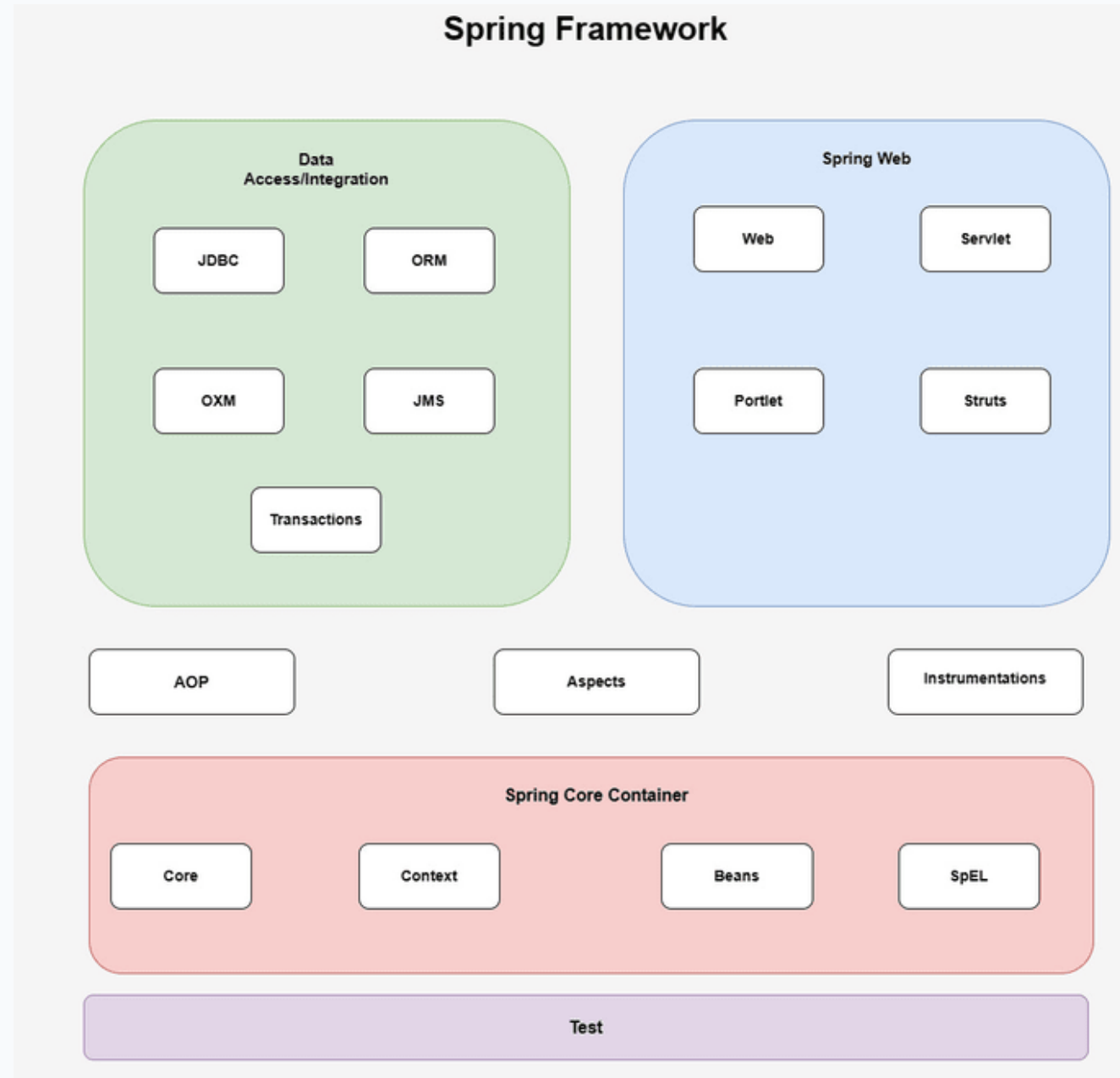
Spring Boot + Spring Web + Spring Data JPA + PostgreSQL



Spring Framework

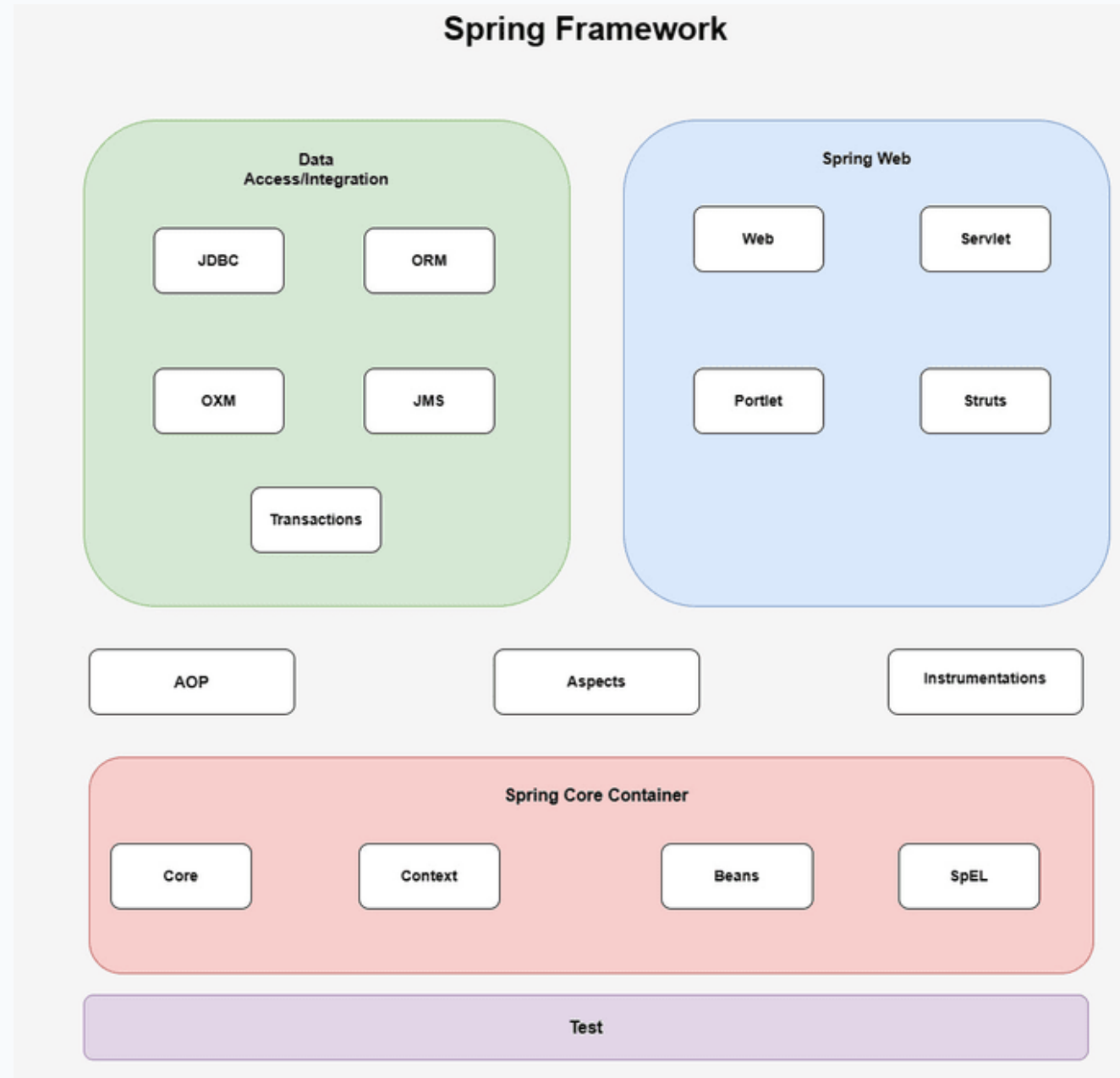
Core – odnosi się do podstawowego modułu kontenera wstrzykiwania zależności.

`org.springframework.beans` i `org.springframework.context` tworzą fundamenty dla zarządzania obiektami i kontekstem w aplikacjach Spring



Spring Framework

Context aplikacji Spring zarządza instancjami beanów, które są tworzone zgodnie z definicjami zadeklarowanymi przez programistę w konfiguracji aplikacji (XML, adnotacje, Java config). Kontekst zarządza cyklem życia beanów, wstrzykując zależności, zarządzając ich tworzeniem.



Spring Framework

Przykład użycia kontekstu aplikacji
Spring MVC w aplikacji webowej.

Servlet jest podstawowym elementem aplikacji biznesowych tworzonych w języku Java, która jest zdolna do przetwarzania żądań HTTP. Servlety wykorzystywane są zarówno w aplikacjach Javy EE jak i tych tworzonych w Spring MVC.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
  id="WebApp_ID" version="4.0">
  <display-name>mvceexample</display-name>
  <!-- Ustawiamy Dispatcher Servlet na ten który udostępnia nam Spring
Framework -->
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <init-param>
      <!-- Ustawiamy kontekst aplikacji: -->
      <param-name>contextConfigLocation</param-name>
      <param-value>/WEB-INF/mvccontext.xml</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <!-- mapujemy url: -->
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>/</url-pattern>
  </servlet-mapping>
</web-app>

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:context="http://www.springframework.org/schema/context"
  xmlns:mvc="http://www.springframework.org/schema/mvc"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd
http://www.springframework.org/schema/mvc
http://www.springframework.org/schema/mvc/spring-mvc.xsd
http://www.springframework.org/schema/mvc ">
  <context:component-scan base-package="org.example"/>
  <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/view/" />
    <property name="suffix" value=".jsp" />
  </bean>
</beans>
```

Spring Framework

Aplikacja napisana z wykorzystaniem SpringMVC musi zostać umieszczona w Kontenerze aplikacji webowej.

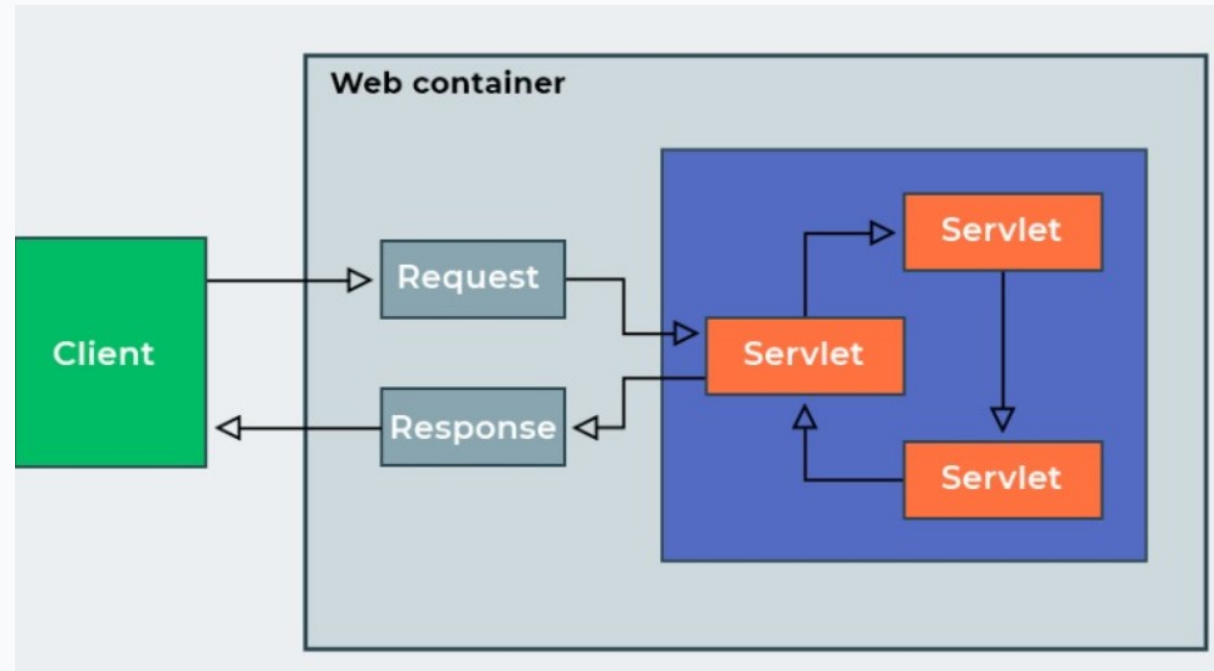
Funkcje kontenera:

1.przetwarza żądanie

2.Dynamicznie generuje strony HTML z plików JSP.

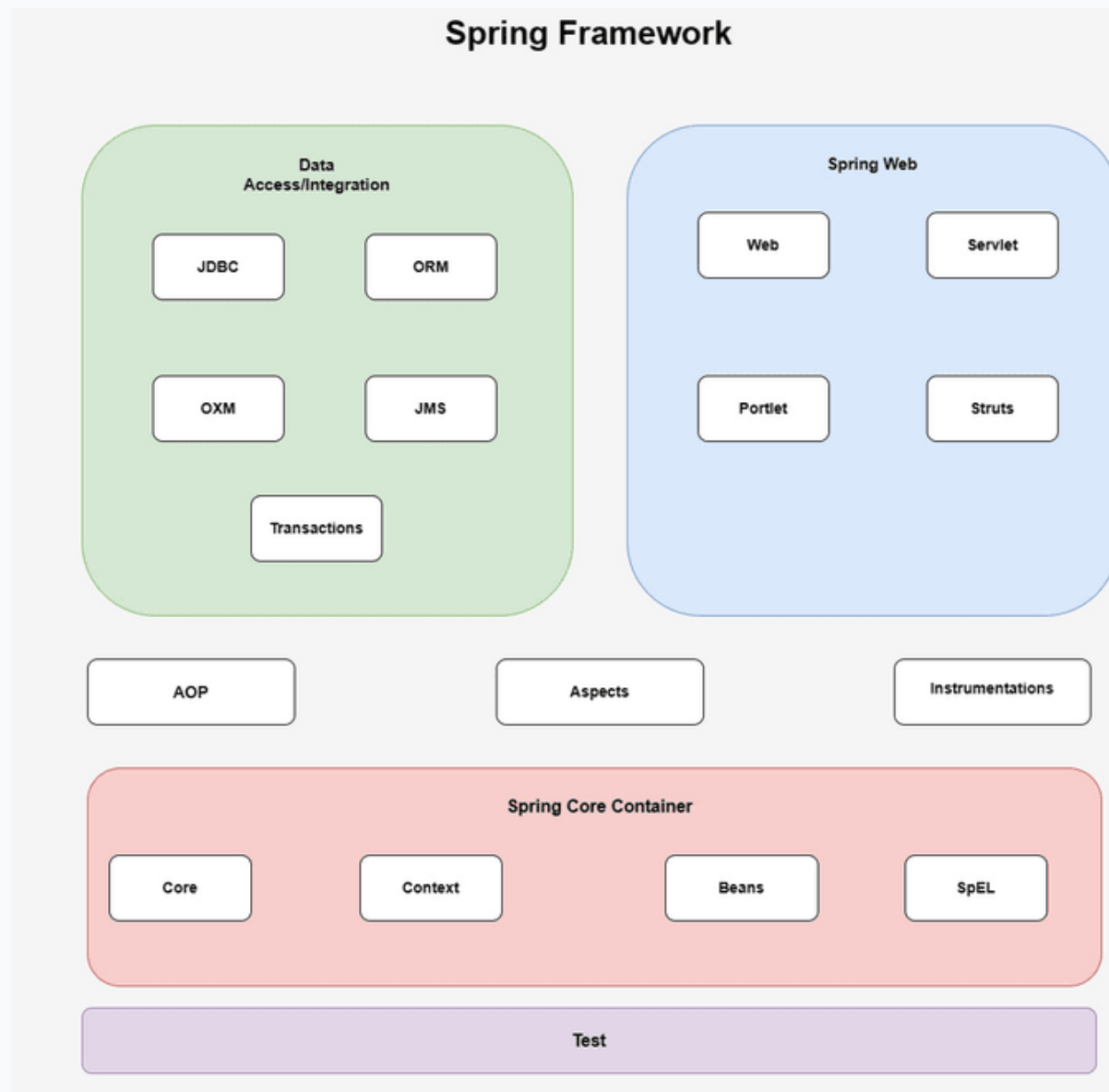
3.Szyfruje/odszyfrowuje wiadomości HTTPS.

4.Zapewnia ograniczony dostęp do administrowania servletami.



Spring Framework - Data Access

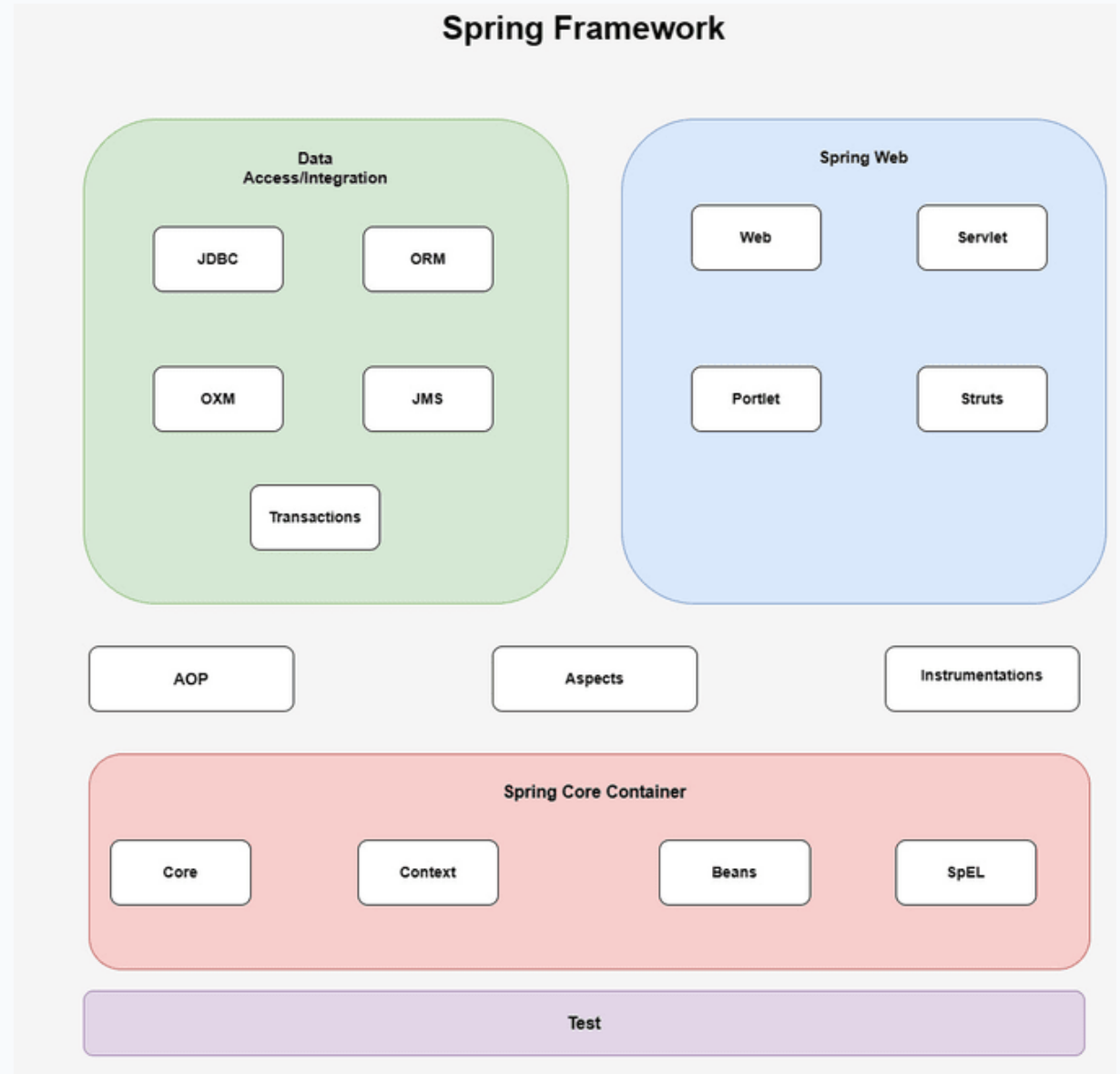
Umożliwia programistom korzystanie z interfejsów API persystencji, takich jak JDBC i Hibernate



Spring Framework - Spring Web

Spring MVC Umożliwia budowanie aplikacji internetowych w oparciu o architekturę MVC. Wszystkie żądania użytkownika przechodzą najpierw przez kontroler, a następnie są przesyłane do różnych widoków wraz z danymi z modelu, takich jak stron JSP, lub Thymeleaf.

Spring Web umożliwia także tworzenie aplikacji REST(Representational State Transfer).



Projekty Springa



Spring Boot

Takes an opinionated view of building Spring applications and gets you up and running as quickly as possible.



Spring Framework

Provides core support for dependency injection, transaction management, web apps, data access, messaging, and more.



Spring Data

Provides a consistent approach to data access – relational, non-relational, map-reduce, and beyond.



Spring Cloud

Provides a set of tools for common patterns in distributed systems. Useful for building and deploying microservices.



Spring Cloud Data Flow

Provides an orchestration service for composable data microservice applications on modern runtimes.



Spring Security

Protects your application with comprehensive and extensible authentication and authorization support.



Spring Authorization Server

Provides a secure, light-weight, and customizable foundation for building OpenID Connect 1.0 Identity Providers and OAuth2 Authorization Server products.



Spring for GraphQL

Spring for GraphQL provides support for Spring applications built on GraphQL Java.



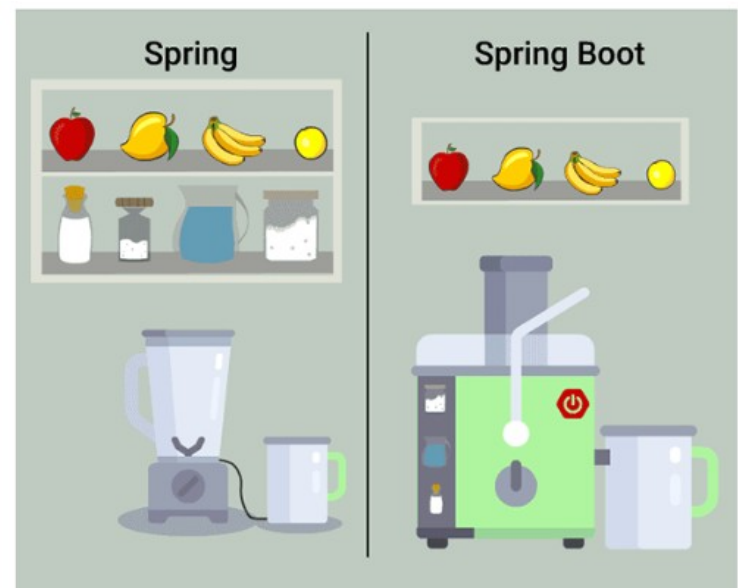
Spring Session

Provides an API and implementations for managing a user's session information.



Spring Integration

Supports the well-known Enterprise Integration Patterns through lightweight messaging and declarative adapters.



The Difference Between Spring vs. Spring Boot

Projekty Springa

Spring jest lekki i modułowy.
Obecnie zależności ekosystemu można „wyklikać” używając SpringInitializr:

The screenshot shows the Spring Initializr web application interface. The browser address bar displays 'start.spring.io'. The interface is dark-themed and includes a sidebar with a hamburger menu icon. The main content area is divided into several sections:

- Project:** Includes radio buttons for 'Gradle - Groovy', 'Gradle - Kotlin', 'Java' (selected), and 'Groovy'. Below this is a 'Maven' radio button.
- Spring Boot:** Includes radio buttons for '3.5.0 (SNAPSHOT)', '3.5.0 (RC1)', '3.4.6 (SNAPSHOT)', '3.4.5' (selected), '3.3.12 (SNAPSHOT)', and '3.3.11'.
- Project Metadata:** Includes input fields for 'Group' (com.umcsuser), 'Artifact' (car-rent), 'Name' (car-rent), 'Description' (Car Rent project), and 'Package name' (com.umcsuser.car-rent). It also has 'Packaging' radio buttons for 'Jar' (selected) and 'War', and 'Java' version radio buttons for '24', '21', and '17' (selected).
- Dependencies:** A section with a button 'ADD DEPENDENCIES... CTRL + B'. It lists several dependencies with their categories in green boxes: 'Spring Web' (WEB), 'Spring Data JPA' (SQL), 'Lombok' (DEVELOPER TOOLS), 'PostgreSQL Driver' (SQL), 'Spring Boot DevTools' (DEVELOPER TOOLS), and 'Validation' (I/O). Each dependency has a brief description.

At the bottom of the interface, there are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and an ellipsis button '...'.

Spring Boot jest projektem w ekosystemie Spring, ułatwiającym tworzenie aplikacji Spring, minimalizując potrzebę konfiguracji i zarządzania zależnościami.

Spring Framework a Spring Boot

Spring Boot nie zastępuje Springa - buduje aplikację na jego podstawie

Spring Framework	Spring Boot
• kontener IoC i dependency injection • Spring MVC / Web • transakcje • integracja z JPA, JDBC, Hibernate • wiele modułów i możliwości konfiguracji	• startery zależności • autokonfiguracja • wbudowany serwer aplikacji • łatwy start z main() • konfiguracja przez application.properties

Spring

Spring Boot

The Difference Between Spring vs. Spring Boot

Kontener i beany

Obiekty aplikacji są tworzone i łączone przez Springa

Bean

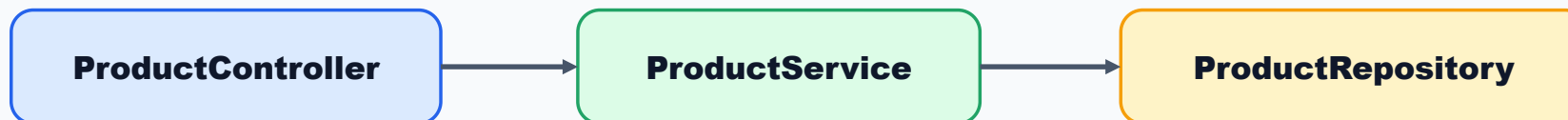
obiekt zarządzany przez Springa, np. ProductController, ProductService, ProductRepository

Kontener

mechanizm, który tworzy beany, przechowuje je i wstrzykuje zależności do konstruktorów

Dependency Injection

klasa nie tworzy zależności przez new, tylko dostaje je z zewnątrz



Hibernate

Typowy schemat z poprzednich zajęć:

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();

Product product = session.get(Product.class, id);

tx.commit();
session.close();
```

W Spring Boot ta sama idea jest ukryta za repozytorium i transakcją:

```
@Transactional(readOnly = true)
public Optional<Product> findById(Long id) {
    return productRepository.findById(id);
}
```

Efekt

Mniej kodu. Więcej logiki aplikacji i endpointy HTTP.

Start aplikacji Spring Boot

Jedna klasa uruchamia kontekst Springa i serwer HTTP

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

- @SpringBootApplication oznacza główną konfigurację aplikacji
- Spring skanuje pakiet com.example.demo i podpakiety
- uruchamia wbudowany serwer na skonfigurowanym porcie
- tworzy beany: controller, service, repository

W demo

Klasy Product, ProductRepository, ProductService i ProductController są w tym samym pakiecie, więc Spring je znajdzie.

Projekt demo: minimalny zestaw zależności

Web + JPA + PostgreSQL wystarczą do prostej aplikacji REST

W projekcie potrzebujemy tylko kilku elementów:

spring-boot-starter-web

kontrolery REST, JSON, obsługa HTTP, wbudowany serwer

spring-boot-starter-data-jpa

Spring Data JPA, repozytoria, integracja z Hibernate

postgresql

sterownik JDBC do połączenia z bazą PostgreSQL

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

Konfiguracja: application.properties

Najważniejsze ustawienia aplikacji są w jednym pliku

```
spring.application.name=demo
server.port=8080

spring.datasource.url=${DB_CONNECT_URL}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

server.port=8080

adres lokalny aplikacji: <http://localhost:8080>

ddl-auto=update

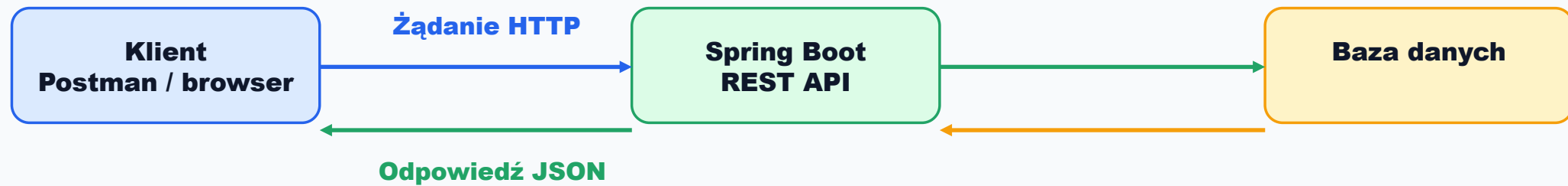
Hibernate może dopasować tabelę do encji

show-sql=true

w konsoli widać zapytania SQL generowane przez Hibernate.

HTTP:

Aplikacja webowa odpowiada na żądania klienta



GET
odczyt

POST
utworzenie

PUT
aktualizacja

DELETE
usunięcie

REST API w demo Product

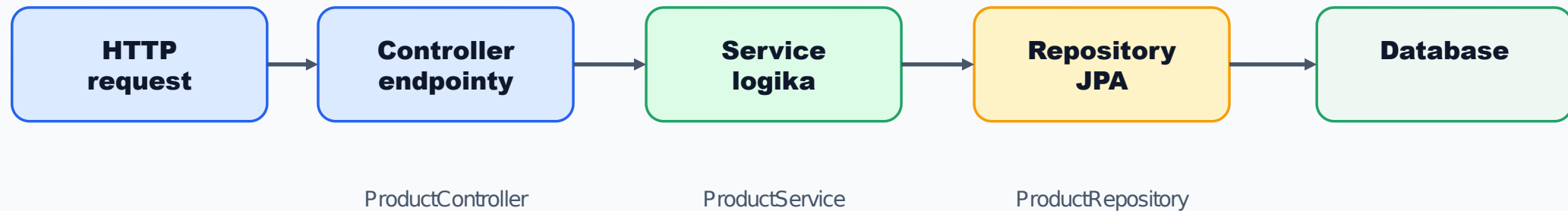
Każdy endpoint odpowiada jednej operacji na produkcie

Metoda	URL	Znaczenie
GET	/api/products	lista wszystkich produktów
GET	/api/products/active	lista aktywnych produktów
GET	/api/products/{id}	jeden produkt
POST	/api/products	dodanie produktu
PUT	/api/products/{id}	aktualizacja produktu
DELETE	/api/products/{id}	usunięcie produktu

Adres lokalnie: <http://localhost:8080/api/products>

Warstwy aplikacji

Żądanie przechodzi przez kilka prostych elementów



Jedna odpowiedzialność

Kontroler obsługuje HTTP, serwis zawiera logikę, repozytorium komunikuje się z bazą.

Encja JPA: Product

Klasa Java, która reprezentuje rekord w tabeli products

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120)
    private String name;

    @Column(nullable = false)
    private BigDecimal price;

    @Column(nullable = false)
    private boolean active = true;
}
```

- @Entity - klasa jest encją JPA
- @Table(name = "products") - nazwa tabeli
- @Id - klucz główny
- @GeneratedValue - id generowane przez bazę
- @Column - ograniczenia kolumny

Tabela products

Przy ddl-auto=update Hibernate może utworzyć strukturę z encji

Encja Product

```
id: Long  
name: String  
price: BigDecimal  
active: boolean
```



Tabela w PostgreSQL

kolumna	typ	uwagi
id	bigint	primary key
name	varchar(120)	not null
price	numeric	not null
active	boolean	not null

Repository: Spring Data JPA

Interfejs zamiast własnej klasy DAO

```
@Repository
public interface ProductRepository
    extends JpaRepository<Product, Long> {

    List<Product> findByActiveTrue();
}
```

JpaRepository<Product, Long>

- Product - typ encji obsługiwanej przez repozytorium
- Long - typ pola id w encji Product
- findAll(), findById(), save(), deleteById() są już gotowe
- findByActiveTrue() Spring odczyta z nazwy metody

Różnica względem DAO

Nie piszemy ręcznie SQL ani SessionFactory w każdej operacji. Deklarujemy interfejs, a Spring tworzy implementację.

Nazwy metod jako zapytania

Spring Data JPA potrafi zbudować query na podstawie nazwy metody

Metoda w repozytorium	Sens zapytania
<code>findByActiveTrue()</code>	<code>WHERE active = true</code>
<code>findByName(String name)</code>	<code>WHERE name = ?</code>
<code>findByPriceGreaterThan(BigDecimal p)</code>	<code>WHERE price > ?</code>
<code>findByActiveTrueAndNameContaining(String text)</code>	<code>WHERE active = true AND name LIKE %text%</code>

Dokumentacja: <https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html>

Service: logika aplikacji

Serwis oddziela kontroler od dostępu do danych

```
@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    @Transactional(readOnly = true)
    public List<Product> findAll() {
        return productRepository.findAll();
    }
}
```

- @Service klasa jest beanem warstwy logiki
- constructor injection: ProductRepository jest wstrzykiwane przez Springa
- kontroler nie powinien bezpośrednio decydować o szczegółach zapisu
- serwis jest dobrym miejscem na transakcje i reguły biznesowe

@Transactional

Spring zarządza transakcją zamiast ręcznego begin/commit/rollback

```
@Transactional(readOnly = true)
public Optional<Product> findById(Long id) {
    return productRepository.findById(id);
}

@Transactional
public Product create(Product product) {
    product.setId(null);
    product.setActive(true);
    return productRepository.save(product);
}
```

readOnly = true

Metoda tylko odczytuje dane. To sygnał dla mechanizmu transakcji i ORM.

@Transactional

Dla zapisu Spring otwiera transakcję, a po zakończeniu metody zatwierdza albo wycofuje zmiany.

Ważne

W metodzie update nie ma save(), bo encja pobrana z repozytorium jest zarządzana przez JPA w transakcji.

Controller: wejście od strony HTTP

Kontroler mapuje adresy URL na metody Java

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService
productService) {
        this.productService = productService;
    }
}
```

- @RestController - odpowiedź metody jest zwracana jako JSON
- @RequestMapping("/api/products") - wspólny początek adresu
- ProductService jest wstrzykiwany przez konstruktor
- metody w kontrolerze są mapowane przez GET, POST, PUT, DELETE



GET: odczyt produktów

Najprostsze endpointy zwracają listę albo pojedynczy obiekt

```
@GetMapping
public List<Product> getAllProducts() {
    return productService.findAll();
}

@GetMapping("/active")
public List<Product> getActiveProducts() {
    return productService.findActive();
}

@GetMapping("/{id}")
public ResponseEntity<Product> getProductById(@PathVariable Long id) {
    return ResponseEntity.of(productService.findById(id));
}
```

Adresy:

```
GET /api/products
GET /api/products/active
GET /api/products/1
```

ResponseEntity.of(...)

Jeżeli Optional ma produkt - zwróci 200 OK. Jeżeli Optional jest pusty - zwróci 404 Not Found.

Wynik

Spring automatycznie zamienia obiekty Product na JSON.

POST: dodawanie produktu

Dane z body żądania są zamieniane na obiekt Product

```
@PostMapping
public ResponseEntity<Product> createProduct(@RequestBody Product
product) {
    Product savedProduct = productService.create(product);
    return
    ResponseEntity.status(HttpStatus.CREATED).body(savedProduct);
}
```

```
@Transactional
public Product create(Product product) {
    product.setId(null);
    product.setActive(true);
    return productRepository.save(product);
}
```

Przykład JSON do Postmana:

```
{
  "name": "Laptop",
  "price": 3999.99
}
```

201 Created

Status HTTP informuje, że nowy zasób został utworzony. W odpowiedzi wraca zapisany produkt z wygenerowanym id.

PUT: aktualizacja produktu

Id jest w adresie, nowe dane są w body żądania

```
@PutMapping("/{id}")
public ResponseEntity<Product> updateProduct(
    @PathVariable Long id,
    @RequestBody Product product
) {
    return productService.update(id, product)
        .map(ResponseEntity::ok)
        .orElseGet(() -> ResponseEntity.notFound().build());
}
```

```
@Transactional
public Optional<Product> update(Long id, Product newData)
{
    return productRepository.findById(id)
        .map(product -> {
            product.setName(newData.getName());
            product.setPrice(newData.getPrice());
            product.setActive(newData.isActive());
            return product;
        });
}
```

Przykład:

```
PUT /api/products/1
{
  "name": "Laptop Pro",
  "price": 4299.99,
  "active": false
}
```

DELETE: usuwanie produktu

Najkrótszy endpoint w demo

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteProduct(@PathVariable
Long id) {
    productService.deleteById(id);
    return ResponseEntity.noContent().build();
}
```

```
@Transactional
public void deleteById(Long id) {
    productRepository.deleteById(id);
}
```

Endpoint:

DELETE /api/products/1

204 No Content

Operacja zakończona sukcesem, ale odpowiedź nie zawiera body.

Alternatywa

Soft delete: product.setActive(false). Wtedy GET /active ukrywa produkt, ale rekord zostaje w tabeli.

Postman: przykłady żądań

Te same operacje można wykonać bez frontendu

#	Metoda	URL	Body
1	GET	http://localhost:8080/api/products	brak
2	POST	http://localhost:8080/api/products	{ "name": "Laptop", "price": 3999.99 }
3	GET	http://localhost:8080/api/products/1	brak
4	PUT	http://localhost:8080/api/products/1	{ "name": "Laptop Pro", "price": 4299.99 }
5	DELETE	http://localhost:8080/api/products/1	brak

Header dla POST/PUT: Content-Type: application/json

Przykładowy JSON

Spring automatycznie serializuje obiekty Java do odpowiedzi HTTP

Po POST:

```
{
  "id": 1,
  "name": "Laptop",
  "price": 3999.99,
  "active": true
}
```

Po GET /api/products:

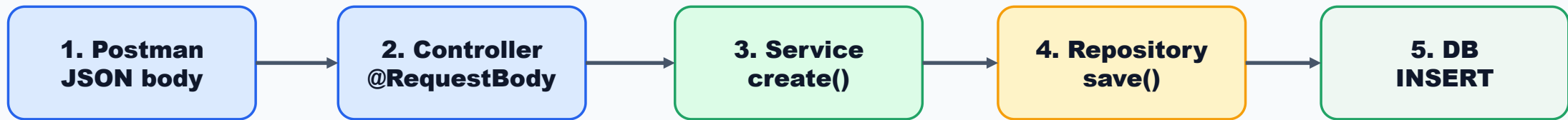
```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 3999.99,
    "active": true
  }
]
```

JSON

Jackson zamienia obiekty Product na JSON, a @RequestBody potrafi zamienić JSON z żądania na obiekt Product.

Co dzieje się po wysłaniu żądania?

Przepływ dla POST /api/products



Odpowiedź: 201 Created + Product jako JSON

```
POST /api/products -> ProductController.createProduct() -> ProductService.create() -> ProductRepository.save()
```

Co Spring Boot zrobił automatycznie?

Skanowanie komponentów

@RestController, @Service, @Repository

Wbudowany serwer

aplikacja działa pod localhost:8080

JSON

@RequestBody i odpowiedzi z obiektów Java

JPA / Hibernate

EntityManagerFactory, repozytoria, SQL

Transakcje

@Transactional wokół metod serwisu

Konfiguracja

application.properties zamiast wielu plików XML

Podsumowanie

- Spring Boot upraszcza konfigurację aplikacji Spring.
- `@SpringBootApplication` uruchamia aplikację oraz włącza automatyczną konfigurację i skanowanie komponentów.
- `@RestController` obsługuje żądania HTTP i mapuje je na metody w Javie.
- `@Service` zawiera logikę biznesową i wyznacza granice transakcji
- `@JpaRepository` udostępnia gotowe operacje CRUD bez konieczności pisania DAO.
- `@Entity` mapuje klasę na tabelę w bazie danych, a JPA/Hibernate zarządza zapisem i odczytem danych.
- Postman pozwala sprawdzić API bez frontendu.



Dziękuję za uwagę

Spring

